

Using the CREED lab*

Updated September 2026

Check for latest document

Tell me if anything is unclear, wrong or missing so we can update it!

Contents

1	Experimental Timeline	2
1.1	Overview	2
2	Running Experiments in the Lab	2
2.1	Reserving the Lab	2
2.2	CREED Website - Managing Participant Sign-up	3
2.3	Running in the Lab	4
2.4	Saving Lab Session Data	5
3	Paying Participants	5
3.1	Cash	5
3.2	Digital Transfer (UvA Administration)	5
4	oTree Template	7
5	FAQ	8
5.1	Running the Study Online	8
5.2	Lab Computers	8
A	Appendix	9
A.1	Custom PostgreSQL Setup	9
A.2	oTree Tips and Tricks	9

*This is unofficial and merely what I jotted down so I would remember it next time I want to run. All mistakes are my own. J. Tait

1 Experimental Timeline

1.1 Overview

Before running an experiment at the CREED Lab or applying for ACBC funding, present your project at the CREED Lunch Seminar. Contact Julian Kirschner or Madgalena Wasilewska to schedule.

Before running, ensure you have each of the following:

1. **Pre-analysis plan:** AsPredicted
2. **UvA/EB Project:** Research Management Services
 - Ethics review
 - Data Management Plan
3. **Funding:** for example from ACBC

Pre-Analysis Plan Questions.

1. Has any data already been collected?
2. What is the main question or hypothesis?
3. Key dependent variable(s) and measurement method?
4. Number and nature of conditions?
5. Planned analyses?
6. Outlier handling and exclusion criteria?
7. Sample size or rule?
8. Additional info?

2 Running Experiments in the Lab

We use the CREED website to manage everything about participant sign-ups. In the lab you run your experiment from your oTree folder on the local server. You need to make sure you i) reserve the lab, ii) get the participants invited, and iii) your code works on all the computers in the lab. Find instructions on how to make sure this all works below.

2.1 Reserving the Lab

Coordinate with Arthur Schram to reserve lab access and receive credentials for CREEDExperiment.nl.

2.2 CREED Website - Managing Participant Sign-up

Getting Started with the Website.

- Login details: from Joep
- Experiment number: from Ailko
- URL: <http://www.creedexperiment.nl/proftulp/loginexp.html>
- Create a new session via “new exp”

Managing Experimental Sessions.

- Modify session: Show Session > Modify
- Send reminder: Show Session > Reminder
- Finish session: Show Session > Finish

Inviting Participants.

1. click 'make a table with potential subjects with name and email address'
2. click 'send emails to a limited number of subjects (like 100)'
3. Prepare the customisable email template
4. Limit email sends (e.g., 100 per batch)
5. System tracks sent emails and removes people from the table to avoid re-sending to the same participant.

Only create new tables the first time you send invites or when you don't have enough 'mails to send' in the screen attached below. Once you create a table it removes anyone you have already sent a mail to from the table, hence why the number decreases. This avoids sending duplicates.

You have still **295** mails that you can send.

I got a complaint from the provider (Feb 10, 2022) that their mailserver was overloaded and mails were delayed. Please, after you have send 100 mails or so, wait a few minutes before sending the next batch. We also should avoid flooding the students with announcements, especially when more experiments are run. Today already 456 mails have been sent. If that wasn't you, other experimenters are also sending mails!

Number:	Send the email to 100 participants.
Message: PLEASE EDIT when you have already run sessions!!	We invite you to participate in CREED Experiment (2418) on economic decision making. The experiment will take about 1 hour. Mon 04 Nov 2024, 11:00 (20 participants needed) Your reward will depend on your choices (usually between 15 and 25 euros). Enlist on www.creedexperiment.nl Best regards, CREED
Send also the first email to myself:	
	Announce Reset

[logout](#)
[new exp](#)
[open exp](#)
[back](#)

Sending Reminders. Navigate to Show Sessions, select a session, and click **Reminder**.

- You can customise the reminder (subject, message).
- Add your email to test delivery.
- Double-check it's for the right session before sending.

Closing the Session. After running a session you need to log attendance. Navigate to Show Sessions and 'Finish' the relevant session. You can then select who showed up and who didn't as well as log how the session went.

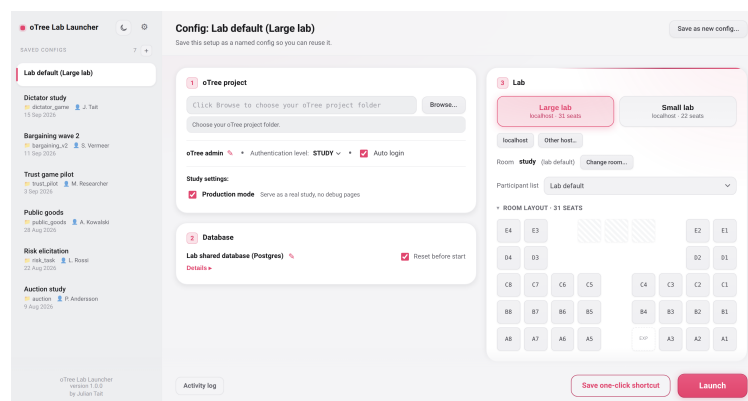
2.3 Running in the Lab

Put your oTree project on the lab computer, in a folder on the desktop identified with your name.

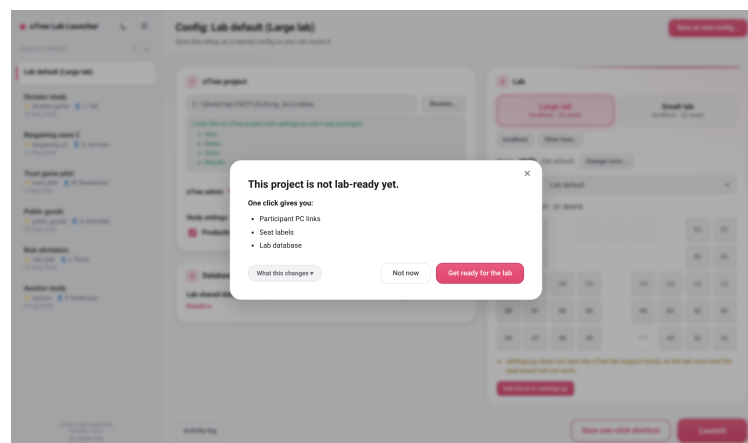
1. Double-click **Start oTree LabLauncher** on the desktop.



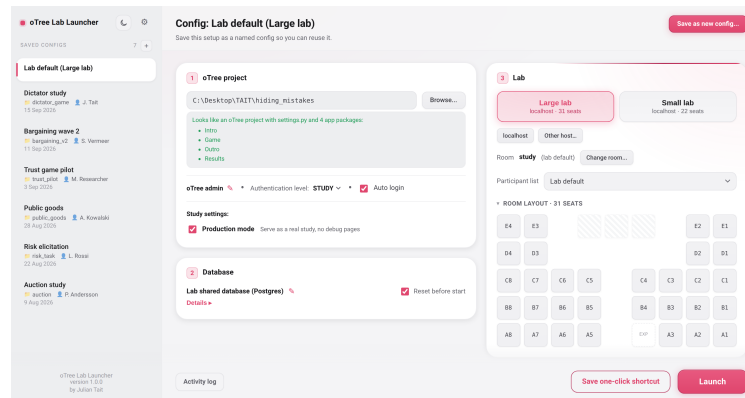
2. Click **Browse** and point it at your oTree study folder.



3. Get it ready for the lab. In one step this wires up the participant computer seat links, the seat labels, and the shared lab database.



4. Launch. This starts the server and makes each lab computer open its own seat link.



5. Optional: Save a one-click shortcut to the desktop for next time.

Monitor the Session. Once launched, you watch the seats fill on the monitor: each station starts grey and turns green when someone arrives at it. When the seats you need are green the session is live and you are ready to begin.

2.4 Saving Lab Session Data

Download it from oTree after each session is completed in the 'Data' tab. Save it in some data folder in your project folder as 'Session_XX.csv'. This ensures that whatever happens to the database on the experimenter computer you already have your data elsewhere.

3 Paying Participants

3.1 Cash

Ask your supervisor to email Ailko (ailko@xs4all.nl) with:

- Experiment number
- Total cash needed
- Preferred denominations

Bring them up one-by-one to pay them the cash in the correct amounts. Make sure they sign a receipt of payment so you can prove where the cash has gone.

3.2 Digital Transfer (UvA Administration)

Overview: Participants can be paid via bank transfer from the central UvA administration (SEPA only). If their IBAN is not in the SEPA region, arrange an alternative (e.g. cash to be collected from your office).

1. Collect Payment Details

- Collect **IBAN** from all participants.
- If IBAN is not Dutch, also collect the **BIC/SWIFT** code.
- Check if IBAN belongs to a SEPA country.

- If not SEPA, show: *“Please speak to the experimenter before leaving.”*

2. Handle Non-SEPA Participants

- Inform them that **cash payment** will be arranged later.
- Record their contact details and agree on an office pickup time.
- You can enter your bank details into the payment schedule and source the cash privately. This proved a very small share of total participants (6 of 350).

3. Submit SEPA Bank Payments

- Compile valid IBANs in the required format (download link)
- Email `servicedesk-ac@uva.nl` with the payment list and request processing.

4. Track Payment Status (& maintain a good reputation for CREED)

- Look for two emails: *confirmation of processing* and *confirmation of payment*.
- Keep a log of any **rejected IBANs** and the **session** that participant was in so you can identify people who reach out to CREED.
- Email a list of the failed payments to `experiment@creedexperiment.nl` with subject 'Failed Transfers ExpXXXX' so the data is available to anyone checking the email (**ideally you!**). This should identify a participant with information on: i) amount, ii) session + date, iii) failed IBAN, and iv) experiment number

4 oTree Template

There is an oTree template app that comes with a lot of the functionality you need to run an experiment in the lab, so you can start from a working structure instead of a blank project. The idea is that you drop your own game into it and reuse everything around it.

It is built in four parts:

- **before** (welcome + consent)
- **intro** (instructions + quiz)
- **main** (main experimental game/task)
- **outro** (demographics + payment)

Out of the box it gives you:

- A holding page so participants cannot start before you release the session.
- Instructions that paginate automatically, with a quiz that reads its questions from a single file and lets participants go back if they answer wrong.
- Treatment assignment handled at session creation.
- Demographics collection (including IBAN + BIC) and a built-in results/payment summary.
- A configurable payment rule for how payoffs are calculated.
- Helper scripts to start oTree in the lab and to turn raw session data into payment and anonymised data files.

The template now ships an experimenter dashboard, a live, read-only monitoring page for a running session. You open it at `/experimenter_dashboard` (or `/experimenter_dashboard/<session_code>`) behind oTree's own admin login to watch the session at a glance (who has arrived, and how far each participant has got), and it refreshes automatically. It is strictly read-only and cannot affect participants.

Note: The lab office computers have no internet access. Any font, Python, or JavaScript/Java dependency not already installed there must be downloaded in advance and placed inside your experiment folder so it is linked locally, not fetched at runtime.

You can download the template and read the full details on how to use and edit it at github.com/juliantait/Template.

5 FAQ

5.1 Running the Study Online

If you run the study online rather than in the lab, you have two hosting options:

- **Railway (recommended).** Railway gives you one customizable URL and lets you run two cheap servers, with server capacity that is easy to diagnose and upgrade as needed, unlike the more cumbersome Heroku. See the [Hosting on Railway](#) guide for the full how-to.
- **Heroku (older fallback).** Heroku still works but is more cumbersome to scale and diagnose. As a rough guide a standard setup handles around 20–50 participants at once; going higher means paying for larger Postgres, Redis, and extra web and worker dynos. If wait pages stop refreshing, set `PGSSLMODE=require` (`heroku config:set PGSSLMODE=require`).

5.2 Lab Computers

Q: Why does my experiment work on my laptop but not on the lab computers?. The lab office computers have no internet access, so any font, Python, or JavaScript/Java dependency not already installed there will fail. Download the required packages in advance and place them inside your experiment folder so they are linked locally rather than fetched at runtime.

A Appendix

A.1 Custom PostgreSQL Setup

If you want to set up your own PostgreSQL instead of using section 2.3 then you need to follow these steps. If you follow the steps above `set_up_otree.bat` you will automatically run with **PostgreSQL**. If not, you should know that oTree defaults to a basic database called SQLite. This can lag more and does not back up your data in case the computer crashes. I recommend you ensure you're using PostgreSQL when running sessions to avoid problems.

If you download your session data after each session and are happy with other wiping the database after every session (ensuring you already have it saved, see 2.4) you can use an existing PostgreSQL database to read and write your data throughout the session (this is already specified to start in `set_up_otree.bat`). Alternatively, you can also create your own database using i) the code below, or ii) simply opening 'pgAdmin 4' on the experimenter computer and following the steps below.

Using *pgAdmin 4*. This is the simplest way with a visual interface.

1. Create a new user and password:
 - Open pgAdmin.
 - Right-click Login/Group Roles $\rightarrow$ Create $\rightarrow$ Login/Group Role.
 - Set:
 - Name: newname
 - Password tab: enter newpassword
 - Privileges tab: check **all** boxes`
2. Create the database:
 - Right-click Databases $\rightarrow$ Create $\rightarrow$ Database
 - Name it experimentname
 - Set owner to newname

Using the terminal. The master password should be '1234'.

```
psql -U postgres
CREATE DATABASE experimentname;
CREATE USER moi WITH PASSWORD '1234';
GRANT ALL PRIVILEGES ON DATABASE experimentname TO moi;
```

Then set the 'DATABASE_URL' environment variable accordingly in the `set_up_otree.bat` script.

A.2 oTree Tips and Tricks

Some solutions to my oTree problems may be useful to you too. Here's a collection of ones I imagine most people can face.

Determining Experiment Payoff (Using a PayoffSummary Page). Because oTree does not allow retrieving payoffs from past apps directly, each app must store its payoff information in a participant-level variable (e.g., `participant.payoff_vector`). Later apps (e.g., an outro/payment app) can then access this list to compute payments or randomly select paid rounds.

- **Declare the participant variable in settings.py.** Add `payoff_vector` to `PARTICIPANT_FIELDS` so it persists across apps.

```
PARTICIPANT_FIELDS = [
    ...,
    "payoff_vector",
]
```

Listing 1: Adding payoff_vector to Participant Fields (settings.py)

- **Append this app's round payoffs at the end of the app.** Instead of using a `before_next_page` hook on the final page, finish each app with a page that runs once on the final round and aggregates each participant's payoffs from all rounds in that app.

```
class payoff(Page):
    template_name = 'main/payoff.html'

    def vars_for_template(self):
        return {
            'payoff': cu(self.payoff),
            'round_count': self.round_number,
        }

    @staticmethod
    def before_next_page(player, timeout_happened):
        # On the final round, collect all payoffs into the participant vector.
        if player.round_number == C.NUM_ROUNDS:
            payoff_vector = [pr.payoff for pr in player.in_all_rounds()]
            if not hasattr(player.participant, 'payoff_vector') or player.participant.
            payoff_vector is None:
                player.participant.payoff_vector = []
            player.participant.payoff_vector.extend(payoff_vector)
```

Listing 2: PayoffSummary page to store per-app payoffs into participant.payoff_vector

Notes: Place `PayoffSummary` as the final page in your app's `page_sequence`. This ensures the payoffs are stored once the app is complete.

- **Retrieve the stored payoffs later (e.g., in the payment/outro app).** In the payment-determining app, retrieve the accumulated payoffs safely via `getattr`.

```
payoffs_vector = getattr(p.participant, 'payoff_vector', [])
```

Listing 3: Retrieving the accumulated payoff vector in the payment app

- **Example: randomly select paid rounds from the stored vector.** Once you have `payoffs_vector`, you can compute totals or draw a random subset of rounds for payment.

```
import random

payoffs_vector = getattr(p.participant, 'payoff_vector', [])

if payoffs_vector:
    paid_amount = random.choice(payoffs_vector)
else:
    paid_amount = 0
```

Listing 4: Example: select one random payoff from the stored vector

This approach mirrors the pattern used in common oTree outro/payment templates (e.g., `oTree-template/outro`) and avoids relying on cross-app payoff retrieval.